

Algorithmic design and acknowledgements

Moussa Kassim Martin Ocharo Dalia Ahmed Dupe Ojo Alessia Vignoli
Leonardo Tenori Stefano Cacciatore

Contents

0.1	Package Boundary	1
0.2	Graph construction and community detection	2
0.3	UMAP	2
0.4	openTSNE-Style t-SNE	3
0.5	Landmarking	5
0.6	Autotuning	5
0.7	Backend Honesty	5
0.8	References	6

fastEmbedR is built around a KNN-first design. The package implements native one-call nearest-neighbor routes for CPU, Metal, and optional CUDA builds, then consumes the resulting graph in native UMAP or openTSNE-style t-SNE code. It also exposes those native routes through `precompute_knn()` and accepts plain precomputed host KNN index and distance matrices.

This vignette explains what is implemented, what was removed, and which papers or software projects inspired the current code.

0.1 Package Boundary

fastEmbedR owns its one-call KNN routes, PCA initialization, UMAP, openTSNE-style t-SNE, KNN graph construction, community detection, landmark transforms, and quality metrics. It does not call another R package for nearest-neighbor search. A reusable host KNN may still be passed as a plain list containing **indices** and **distances**; this separates KNN and embedding time for controlled comparisons. A CUDA result from **precompute_knn()** remains device-resident and is reusable directly by the CUDA KNN-input embedding APIs.

The embedding functions also accept plain precomputed KNN matrices, so the core package can be checked without any external nearest-neighbor library:

```
library(fastEmbedR)

x <- scale(as.matrix(iris[, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 10L
idx <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
dst <- matrix(d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(idx)))],
  nrow = nrow(d), byrow = TRUE)

layout <- umap_knn(idx, dst, graph_mode = "fuzzy", n_threads = 1, seed = 1)
dim(layout)
#> [1] 150 2
```

0.2 Graph construction and community detection

`knn_graph()` consumes only neighbor indices and distances after the KNN boundary. Its deliberately small weighting surface contains SNN/Jaccard, inverse-distance, and binary edges. Duplicate directed KNN observations are collapsed into one undirected edge, and graph construction is parallelized by source row.

`graph_cluster()` has three native CPU methods:

1. multilevel Louvain local moving and graph aggregation;
2. Leiden local moving, within-community refinement, and aggregation;
3. Pons-Latapy Walktrap random-walk distances and adjacent-community agglomeration.

Leiden's phase organization follows Traag et al. and is informed by the MIT-licensed NetworkKit implementation. The package uses its own compact graph representation and does not link to NetworkKit. Walktrap is not the earlier sampled-proximity approximation: it computes the published degree-scaled random-walk distance and Ward update. Because its transition matrix is quadratic in the number of vertices, the implementation stops before an unsafe allocation and directs large analyses to Leiden or Louvain. **igraph** is used only as a test oracle, not as a runtime dependency. `cuGraph` is not required.

0.3 UMAP

0.3.1 CPU UMAP

The CPU UMAP path is native C++ and operates directly from a supplied KNN object. The implementation keeps the standard UMAP structure:

1. estimate smooth KNN bandwidths;
2. compute local connectivity offsets;
3. build fuzzy graph weights;
4. symmetrize the graph;
5. precompute `epochs_per_sample`;
6. optimize the two-dimensional layout with UMAP's attractive and repulsive stochastic objective.

The main improvements are engineering improvements, not changes to the mathematics:

- compact graph buffers;
- CSR/COO-style graph storage;
- no dense graph intermediates;
- fewer avoidable copies between R and C++;
- `float` internal weights where safe;
- parallel row-wise work where deterministic enough;
- native sampled-edge optimization with negative sampling and learning-rate decay.

0.3.2 Independent initialization boundary

`umap_init()` accepts raw data, a KNN object, or prepared UMAP graph state. It returns a `fastEmbedR_umap_initialization` object containing the prepared sparse graph, initial coordinates, resolved parameters, and stage timings. Passing this object to `umap_knn()` reuses the exact graph and coordinates. This boundary is intended for controlled backend comparisons and repeated optimization; it is not a different UMAP objective.

0.3.3 graph_mode

UMAP has two graph modes:

```
y_binary <- umap_knn(knn, graph_mode = "binary")
y_fuzzy <- umap_knn(knn, graph_mode = "fuzzy")
```

`graph_mode = "fuzzy"` uses the standard UMAP fuzzy graph. `graph_mode = "binary"` assigns unit weight to the symmetric KNN union. Binary mode is an adjacency-only sensitivity analysis for cases where neighbor identity is trusted more than distance calibration; it is not standard UMAP or a guaranteed acceleration. Because every binary edge receives the maximum sampling frequency, binary optimization can be slower than fuzzy UMAP on CPU. Its quality effect is dataset-dependent, so benchmark output records the mode and reports fuzzy and binary results separately.

0.3.4 Metal UMAP

The Metal UMAP backend is implemented in Objective-C++/Metal. It does not call Python, Torch, MLX, or `reticulate`.

The public Metal optimizer is the visually validated `atomic_inplace` path. Earlier scheduled, deterministic, and hybrid experiments were removed from the public API because they made the package harder to reason about and produced less reliable plots.

```
y_metal <- umap_knn(knn, backend = "metal", seed = 1)
```

If Metal is not available, this request fails clearly. It is not silently run on CPU and reported as Metal.

0.3.5 CUDA UMAP

The CUDA UMAP backend uses native CUDA kernels when `fastEmbedR` is built with CUDA support. The cleaned public path is the pure atomic CUDA optimizer. Older CUDA hybrid and deterministic variants were removed.

For one-call CUDA embeddings, `fastEmbedR` invokes native FAISS GPU exact search for smaller inputs and RAPIDS cuVS IVF-Flat for larger inputs. The packed KNN buffers stay on the selected CUDA device through graph or affinity construction and optimization. External GPU objects are not materialized by `fastEmbedR`; callers must convert them explicitly to host KNN matrices.

```
y_cuda <- umap(  
  x,  
  n_neighbors = 50,  
  backend = "cuda",  
  graph_mode = "fuzzy",  
  seed = 1  
)
```

0.4 openTSNE-Style t-SNE

Here, **openTSNE-style** refers to algorithmic lineage, not compatibility with the Python package. `fastEmbedR` implements the published sparse t-SNE affinities, early-exaggeration and normal phases, adaptive optimization, FIt-SNE interpolation/FFT repulsion, and fixed-reference transformation. It does not call or port Python `openTSNE`; its R API, defaults, objects, native kernels, and resulting coordinate trajectories are package-specific.

`opentsne_knn()` implements an openTSNE-style t-SNE optimizer from precomputed KNN matrices.

The high-level steps are:

1. convert neighbor distances to perplexity affinities;
2. symmetrize sparse probabilities;
3. initialize the layout, usually with PCA;
4. run early exaggeration;
5. run the normal optimization phase;
6. return the layout plus timing/backend metadata.

```

y_tsne <- opentsne_knn(
  knn,
  init_data = x,
  perplexity = 15,
  backend = "cpu",
  seed = 1
)

```

0.4.1 PCA Initialization

PCA initialization is explicit. Provide `Y_init` or `init_data` when you want it. For t-SNE this initialization is centered and scaled to the small scale expected by t-SNE:

```

pca_fit <- pca(x, ncomp = 2L, opentsne_init = TRUE)
Y_init <- pca_fit$opentsne_init
y_tsne <- opentsne_knn(knn, Y_init = Y_init, perplexity = 15)

```

In MNIST 70k tests, PCA initialization reduced cluster splitting without changing the objective.

0.4.2 CPU FFT-Grid Path

The CPU openTSNE large-data path uses a FIt-SNE-style grid approximation for the repulsive force. The attractive force remains sparse and KNN-based. The old Barnes-Hut path was removed from the public package surface because the FFT-grid path was the useful large-data standard in the benchmarks.

0.4.3 Metal openTSNE

The Metal openTSNE backend is package-native Objective-C++/Metal:

- grid scatter kernels;
- native Metal FFT-grid convolution;
- grid gather kernels;
- sparse attractive force kernels;
- gain, momentum, update, and centering kernels.

MPSGraph FFT was tested as a diagnostic alternative and the standalone diagnostic helpers were removed because they did not improve enough over the package-native FFT path.

```

y_tsne_metal <- opentsne_knn(
  knn,
  Y_init = Y_init,
  perplexity = 15,
  backend = "metal",
  seed = 1
)

```

0.4.4 CUDA openTSNE

The CUDA openTSNE backend uses native CUDA kernels and cuFFT when the package is built with CUDA support:

- device-side grid scatter/gather;
- cuFFT convolution;
- sparse attractive forces;
- optimizer updates on device.

The CUDA path is designed to match the same openTSNE/FIt-SNE formulation used by the CPU and Metal paths.

0.5 Landmarking

Landmarking is an explicit approximation. It is useful for very large data or interactive workflows, but it should be labelled as landmarking in benchmarks.

```
fit_landmark <- landmark_tsne(  
  x,  
  landmarks = 0.5,  
  n_neighbors = 50,  
  perplexity = 15,  
  seed = 1  
)
```

The package separates:

- embedding time for landmarks;
- projection or transform time for non-landmark points;
- quality metrics for the final full layout.

0.6 Autotuning

The default API asks for few parameters, but the code records what it chose.

openTSNE uses opt-SNE-style rules:

- automatic learning rate is `n / early_exaggeration`;
- iteration defaults are chosen by a native helper;
- CPU/small exact runs can use KLD-based stopping;
- large FFT/GPU runs do not hide an expensive CPU KLD monitor.

UMAP uses the KNN distance profile to choose internal defaults such as initialization effort and optimizer settings. If a KNN object is supplied, the KNN graph itself is not silently changed.

0.7 Backend Honesty

The backend rule is simple:

- `backend = "metal"` must run native Metal code or fail;
- `backend = "cuda"` must run native CUDA/cuFFT code or fail;
- CPU fallback is never reported as GPU work.

Every result records the backend and KNN route that actually ran:

```
fit <- umap(x, n_neighbors = 30, backend = "cpu", seed = 1)  
fit$parameters[c("backend", "nn_backend", "graph_mode")]
```

```
sessionInfo()
```

```
#> R version 4.6.0 (2026-04-24)
```

```
#> Platform: aarch64-apple-darwin23
```

```
#> Running under: macOS Sonoma 14.5
```

```
#>
```

```
#> Matrix products: default
```

```
#> BLAS: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
```

```
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version
```

```
#>
```

```
#> locale:
```

```
#> [1] C
```

```
#>
```

```
#> time zone: Africa/Johannesburg
```

```

#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] fastEmbedR_0.99.0 knitr_1.51      rmarkdown_2.31
#>
#> loaded via a namespace (and not attached):
#> [1] digest_0.6.39 fastmap_1.2.0 Matrix_1.7-5  xfun_0.60      lattice_0.22-9
#> [6] float_0.3-3 parallel_4.6.0 htmltools_0.5.9 tinytex_0.60   cli_3.6.6
#> [11] fastPLS_0.99.6 grid_4.6.0 withr_3.0.3 compiler_4.6.0 tools_4.6.0
#> [16] evaluate_1.0.5 Rcpp_1.1.2 yaml_2.3.12 otel_0.2.0     jsonlite_2.0.0
#> [21] rlang_1.3.0

```

0.8 References

Key references are listed in AACR journal style.

1. van der Maaten L, Hinton G. Visualizing data using t-SNE. J Mach Learn Res 2008;9:2579-2605.
2. van der Maaten L. Accelerating t-SNE using tree-based algorithms. J Mach Learn Res 2014;15:3221-3245.
3. Linderman GC, Rachh M, Hoskins JG, Steinerberger S, Kluger Y. Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. Nat Methods 2019;16:243-245.
4. Policar PG, Strazar M, Zupan B. openTSNE: a modular Python library for t-SNE dimensionality reduction and embedding. J Open Source Softw 2019;4:1576.
5. Chan DM, Rao R, Huang F, Canny JF. t-SNE-CUDA: GPU-accelerated t-SNE and its applications to modern data. arXiv 2018;1807.11824.
6. Belkina AC, Ciccolella CO, Anno R, Halpert R, Spidlen J, Snyder-Cappione JE. Automated optimized parameters for T-distributed stochastic neighbor embedding improve visualization and analysis of large datasets. Nat Commun 2019;10:5415.
7. McInnes L, Healy J, Melville J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv 2018;1802.03426.
8. Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs. IEEE Trans Big Data 2021;7:535-547.
9. Blondel VD, Guillaume JL, Lambiotte R, Lefebvre E. Fast unfolding of communities in large networks. J Stat Mech 2008;2008:P10008.
10. Traag VA, Waltman L, van Eck NJ. From Louvain to Leiden: guaranteeing well-connected communities. Sci Rep 2019;9:5233.
11. Pons P, Latapy M. Computing communities in large networks using random walks. J Graph Algorithms Appl 2006;10:191-218.

Software acknowledgements include `uwot`, `Rtsne`, `FAISS`, `RAPIDS cuVS`, `AppleSiliconFFT`, `mlx-vis`, and `annembed`. See `inst/NOTICE`, `inst/ALGORITHMIC_REFERENCES.md`, and `docs/references.md` for provenance and license notes.